数据库系统课程大作业

基于 MongoDB 的在线问卷系统

第二阶段：题目复用与版本管理

姓名：李雨霏

学号：10245501454

项目地址：https://github.com/lyf-666666/ecnu-db-pj1-survey

完成日期：2026 年 11 月

项目完成报告（第一、二阶段）

GitHub 仓库：https://github.com/lyf-666666/ecnu-db-pj1-survey

姓名：李雨霏　　学号：10245501454　　日期：2026 年 10 月

一、项目目标

第一阶段要做的是一个基于 MongoDB 的简化问卷系统，完整走通“注册登录、创建问卷、配置题目和跳转、分享链接、登录后填写、查看统计”这条链路。需求文档里明确要求所有数据放在 MongoDB，不能用关系数据库，也不能套现成的问卷系统。

我的目标不只是把功能做出来，还想让数据结构经得起第二阶段的需求变更。所以这次在集合划分、跳转规则的存储方式、答案的组织上花了比较多时间，代码量反而控制在能读懂的范围。

二、需求完成情况

需求	实现情况
注册、登录、只看自己的问卷	完成，session 保存登录状态，问卷接口都按 ownerId 过滤
创建问卷、标题、说明、匿名设置	完成
问卷列表、发布、截止时间、关闭	完成，状态分 draft、published、closed
分享链接 `/survey/:id`	完成，填写人需要登录
单选、多选、文本、数字四种题型	完成，多选支持最少/最多选择数，文本支持字数，数字支持范围和小数/整数
必答题	完成，服务端和前端都校验
按单选、多选、数字内容跳转	完成，规则存在题目里，没有硬编码
按顺序填写、自动跳转、错误提示、提交	完成，一次显示一题
多人填写、匿名、重复填写	完成，匿名不保存用户 id，重复填写默认允许
整卷统计、单题统计	完成，单选/多选按选项计数，数字算平均值，文本列出内容

第二阶段的需求变更：

需求	实现情况
题目可以保存下来反复使用	完成，每道题都有独立的题目本体，可以被任意多份问卷引用
一个题目可以被多个问卷使用	完成，问卷里的题目指向具体版本，可以选最新版也可以选历史版
创建题目并保存到题库	完成，手写题目会自动建题目本体，也可以手动归入自定义题库
题目可以共享给其他用户	完成，按用户名共享，对方可以使用但不能修改

已发布问卷里的题目不能被修改	完成，接口、版本、快照三层保护
修改题目生成新版本，旧问卷不受影响	完成，改题只新增版本文档
可以查看题目的历史版本	完成，详情页有版本列表和版本来源
可以知道新题是从哪个题改来的	完成，`parentVersionId` 记录父版本，界面显示“由 vN 改出”
可以恢复旧版本	完成，恢复动作本身也生成一个新版本
同一题的不同版本可以同时存在	完成，不同问卷可以各自指向不同版本
可以查看题目被哪些问卷使用	完成，详情页列出问卷、状态和使用版本
题库管理	完成，支持新建、改名、加入、移出、删除
跨问卷的单题统计	完成，按选项文字合并，支持选项、数字、文本三种题型

三、系统设计

技术栈是 Node.js + Express + MongoDB 原生驱动，前端是原生 HTML/CSS/JS，没有构建工具。选原生驱动而不是 Mongoose，是因为这门课的重点是数据库设计，集合、校验器、索引都自己写一遍更清楚。

服务端分路由和服务两层：

```
routes/auth.js            注册登录
routes/surveys.js         问卷和题目管理
routes/public-surveys.js  公开填写
routes/stats.js           统计
routes/questions.js       题目库、版本、共享、跨问卷统计
routes/banks.js           题库管理
services/question-check.js     题目配置校验
services/answer-check.js       答案校验和跳转重放
services/stats.js              聚合统计
services/question-catalog.js   题目本体、版本、权限
services/password.js           scrypt 密码哈希
```

页面一共六个：登录注册、我的问卷、问卷编辑、填写、统计、题目库。页面之间只通过 JSON 接口交互，没有服务端模板。

权限上只有创建者和填写者两种角色。创建者接口全部要求登录并校验 ownerId；填写接口要求登录，但只返回已发布问卷的公开字段。访问别人的问卷统一返回 404，不暴露是否存在。

四、MongoDB 设计

一共八个集合：`users`、`surveys`、`questions`、`responses`、`question_roots`、`question_versions`、`question_banks`、`bank_items`。前四个是第一阶段，后四个是第二阶段加的，详细字段和取舍见 `docs/02-数据库设计.md` 的 2.9 到 2.14。

题目没有嵌在问卷文档里，而是独立成集合。这样做的好处是修改单道题只动一条文档，跳转规则和回答引用题目 id 也更自然。代价是查问卷详情多一次查询，但题目本来就要按顺序取，联合索引 `{surveyId: 1, order: 1}` 正好覆盖。

回答内嵌在 `responses.answers` 数组里，没有单独建 answers 集合。一次填写是一个整体，一条 insert 写完，统计时 unwind 展开，符合文档模型的使用方式。如果以后要做跨问卷的题目分析，再考虑拆出去。

问卷不存题目数和回答数，列表页用分组聚合现算。存冗余计数要维护多个写入路径，容易不一致，课程作业的数据量下没必要。

四个集合都写了 JSON Schema 校验器，约束类型、枚举、长度和必填字段。业务规则（比如跳转目标必须存在、最少选择数不能大于最多选择数）放在应用层，因为要跨文档检查。

索引建了六个：用户名唯一索引、问卷创建者加时间、题目问卷加顺序唯一、回答按问卷和时间、回答按问卷和填写人、回答按问卷和题目 id（多键）。详细设计见 `docs/02-数据库设计.md`。

五、跳转设计

跳转规则存在 `questions.jumps` 数组里，每条规则包含条件和目标题目 id。条件分两类：选项条件和数字条件。选项条件支持“任意一个”和“全部满足”，数字条件支持六种比较运算。

最关键的决定是服务端重放路径。填写页虽然也会算下一题，但提交时服务端从第一题开始，按顺序取答案、校验、查跳转规则，走到哪里算哪里，只保存实际经过的题目。这样客户端改脚本也绕不过去，被跳过的必答题也不会误报。

服务端还记录实际路径 `path`，方便调试。遇到规则成环时用 visited 集合检测，返回错误而不是死循环。删除题目时会清掉其他题目里指向它的规则，避免悬空引用。

六、校验设计

校验分两层。前端在点“下一题”时检查当前题，给即时提示；服务端在提交时完整重走一遍，前端校验不作为安全边界。

题目配置校验在创建和修改时执行：选项至少两个、key 不重复、多选题的最少不大于最多且不超过选项数、文本最少字数不大于最多、数字最小值不大于最大值、跳转目标存在且不是自己、选项条件的 key 属于当前选项。

答案校验按题型分派：单选必须正好一个合法选项，多选不能重复且数量在范围内，文本按字数（用 Array.from 计算，emoji 算一个字），数字必须是有限数、按需检查整数和范围。错误信息尽量具体，比如“至少需要选择 2 个选项”。

七、AI 使用过程

整个项目用 Trae 辅助完成，模型是 DeepSeek。用法是分阶段提需求：先让它分析集合划分，再生成校验器和索引，然后写接口、页面、测试和文档，每一步都先跑一遍再改。

AI 在重复性代码上省了很多时间，尤其是聚合管道、表单渲染和测试脚手架。但它也犯了几个错：跳转最初放在前端算导致必答题误报、测试用字符串查 ObjectId、前端表单读取用了 `form.title` 这种有风险的写法、删除题目没有清理跳转。这些都在开发过程中改掉了，具体记录见 `docs/06-AI 使用日志.md`。

我自己主要做的是方案取舍和验收：为什么题目独立、为什么回答内嵌、为什么发布后锁题目、统计分母用作答人数还是总提交数。AI 给候选，最后选哪个由我定。

八、测试结果

自动测试 25 个用例全部通过，覆盖注册登录、建卷加题、发布锁定、三种跳转、六种非法输入、统计、匿名、关闭、删除清理，以及第二阶段的版本分叉、共享权限、题库管理和跨问卷统计。运行 `npm test` 大约 5 秒完成，测试使用临时 mongod，不影响本机数据。

浏览器手工测试走完了注册、建卷、添加四种题型、配置数字跳转、发布、填写、匿名提交、查看统计，页面没有控制台报错。统计页显示总回答 1，单选选项 1 人 100%，数字题平均 5。第二阶段另外在建好的题目库页面里做了新建题目、从题库选题加入问卷、查看版本历史的验证。

详细用例、输入和实际结果见 `docs/05-测试用例.md`。

九、遇到的问题和解决

1. **被跳过的必答题误报。** 最初服务端按题号顺序检查所有必答题，第 1 题跳到第 5 题时，第 2、3、4 题会被误判为没回答。改成服务端重放路径，只校验实际经过的题目。这是这次改动最大的地方，也让我理解了“跳转不能只做前端展示”。

2. **发布后改题目会让历史数据对不上。** 比如删掉一个已经被选过的选项，历史回答里还留着那个 key。最后的处理是发布后锁定题目，只能改标题、说明、截止时间和匿名设置。这个规则需求里没写，是我自己加的，写在了系统说明里。

3. **删除题目留下悬空跳转。** 删掉第 5 题后，第 1 题还挂着“跳到第 5 题”的规则。解决是删除时用 `$pull` 清理其他题目的 jumps，并加了测试用例。

4. **统计百分比的分母。** 一开始想用总提交数做分母，但有跳转时很多人没做到某道题，会出现所有选项加起来不到 100% 的情况。改成用作答人数做分母，多选的选项计数仍然会大于作答人数，这是正常的，因为一个人可以选多个。

5. 匿名怎么处理。 一开始想保留 respondentId 只是不展示，后来觉得这不叫匿名。改成匿名提交时直接把 respondentId 写成 null，数据库里也查不到是谁。

6. 测试一直查不到响应文档。 接口返回的 responseId 是字符串，直接拿去查 `_id` 永远是 null。改成 `new ObjectId(responseId)` 后正常。这个错误让我意识到接口序列化和数据库类型不是一回事。

7. 前端题型切换后跳转规则错乱。 从单选切换到数字题时，旧的选项条件控件还在，读取时拿不到数字运算符。解决是切换题型先清空规则区再重绘。

十、第二阶段：题目复用和版本管理

10.1 为什么不能靠“复制题目”解决

拿到需求变更的第一反应是：加一个“保存到题库”的按钮，用的时候复制一份到新问卷。这样做两天就能交差，但有两个问题绕不过去：

一是没法回答“这道题被哪些问卷用过”。复制出来的文档和原题没有任何关联，统计也只能各算各的，需求里的“跨问卷统计”直接落空。

二是没法回答“这道题改过几次”。复制之后每次都是新文档，历史断了。

所以最后做的是版本模型：题目本体（`question_roots`）负责“这是同一道题”，版本（`question_versions`）负责“某个时刻的内容”，问卷里的题目（`questions`）负责“这份问卷用了哪一版”。三层各管一件事，需求里那些看起来互相矛盾的要求就能同时成立。

10.2 改题的分叉规则

最容易写错的是“在哪改”。如果只有题目库一个修改入口，用户会发现改了题库之后，已发布问卷没变、草稿问卷也没变，得再去问卷里手动同步一次，很反直觉。

现在的规则是：

- 在题目库改：只加版本，不动任何问卷。适合“先把题改好，以后再用”。
- 在问卷里改：以这份问卷当前用的版本为父版本，加一个新版本，只把这份问卷指过去。适合“这份问卷想换个问法”。
- 点“更新到 vN”：把草稿问卷的题目换成题库最新版。适合“题库里的修改要同步过来”。

第二条是关键。它保证了“同一道题的不同版本同时存在”：A 卷停在 v1，B 卷走到 v3，互不影响，但它们仍然被认作同一道题，统计能合在一起。

10.3 已发布问卷的三层保护

1. 接口层：所有改题、加题、删题、同步的接口都检查 `status === 'draft'`，发布后一律 400。

2. 版本层：问卷里的题目指向具体版本文档，而版本表只插入不更新。改题库只会新增版本，不会改动旧版本文档。
3. 快照层：questions 文档里保存了完整的内容。填写、跳转判断、统计都读快照，不读版本表。

三层里任何一层单独失效，都不至于让已发布问卷的内容变掉。这不是过度设计，而是因为“历史回答必须和题目对得上”这件事一旦出错就没法补救。

10.4 跨问卷统计的口径

按选项编号合并是错的，因为不同版本里同一个编号可能对应不同文字。现在按选项文字合并，并在页面上写清楚“改过文字的选项会被算成两项”。

另一个决定是只统计当前用户名下的问卷。共享给你的题，你能看到自己问卷里的数据，但看不到对方的。这个边界写在接口说明里，避免用户以为能看到全部数据。

10.5 这一阶段踩的坑

版本号重复。 一开始用 root.latestVersionNo + 1 算新版本号，没考虑并发。后来在 {rootId, versionNo} 上加了唯一索引，最坏情况是后写入的请求报错，不会悄悄产生两个 v3。

恢复旧版本。 最初想把 latestVersionId 直接指回旧版本，这样最省事。但这样“恢复”就会让所有用最新版的问卷跟着变，和“旧问卷不受影响”直接冲突。改成复制内容生成新版本，恢复也变成历史的一部分。

切换版本后跳转失效。 新版本的选项 key 可能和老版本不一样，原来的跳转规则会指向不存在的选项。现在同步版本时直接清空 jumps，让用户重配。清空比留着错的好。

重复的 `$set`。 写更新语句时用了 { $set: update, $set: { jumps: [] } }，JavaScript 对象里后一个同名 key 会覆盖前一个，结果题目内容全丢了。这个错误不会报错，只会静默丢字段，是读代码时才发现的。

统计接口 500。 stats.js 里用了 ObjectId 但忘了 require，测试一跑就 500。补上就好了，但这种错误说明新写的服务文件也要过一遍依赖检查。

十一、不足和下一步

- session 存在进程内存，重启服务要重新登录。生产环境应该换 Redis 或 MongoDB 存储。
- 跳转成环只在提交时检测，配置时没有做完整的图检查。
- 题目不能排序，只能按添加顺序。删除后 order 会留空号，不影响填写。
- 统计每道题查一次聚合，题目多了会慢，可以合并成一次按 questionId 分组。
- 题目库列表一次最多返回 200 道题，没有分页，也没有按使用次数排序。

- 跨问卷统计按选项文字合并，改过文字的同义选项不会自动合并。要解决得给选项加稳定的业务编码。
- 共享只支持按用户名逐个添加，没有共享链接，也没有“共享给所有人”。
- 版本只增不删。题目改得特别多时，版本列表会很长，目前没有归档或折叠。
- 没有做并发测试。版本号的唯一索引能兜底，但没有实际复现过两个人同时改同一道题。
- 没做压力测试，所有性能结论都是基于数据量很小的场景。

题目参数是子文档，加字段不用动其他题；跳转规则是数组，加条件类型只需要改判断函数；回答里保留了 type，统计可以按类型分派。需要特别注意的是旧数据的兼容：第二阶段给题目加 `rootId` 时，第一阶段的题目文档里没有这个字段，所以写了迁移脚本，而不是假设数据都是新的。